

Documentation GIT

Organisation générale de git pour ce projet

- Une branche master protégée qui sert pour le rendu. Doit être absolument stable. (Pipeline et doc)
- Une branche develop protégée où merge les features. Doit être stable. Sert à tester l'intégration des feature les une avec les autres
- Les branches de feature doivent être tirés depuis develop et merge sur develop

Création de branche

Créer une branche tirée de develop respectant le pattern suivant:

```
PGL-##/<titre_du_ticket>/<pseudo>
```

par exemple: `PGL-04/implement_operator_plus/wendelsj`

Pour cela utiliser la commande (depuis develop):

```
git checkout -b "PGL-04/implement_operator_plus/wendelsj"
```

En cas de publication de branche qui renvoie un message d'erreur de ce style:

```
canaguyv@ensipc432 in repo: Projet_GL on  cette_branche_est_nulle is  v0.0.1-SNAPSHOT via  v18.0.2 took 15ms
[🔴] × git push --set-upstream origin cette_branche_est_nulle
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote: GitLab: Branch name 'cette_branche_est_nulle' does not follow the pattern '^(master|develop)$|(PGL-[0-9]
+){1}\/(.{5,20})\/(wendelsj|canaguyv|grahamm|laisnmat|materh|guinetna|ngongang|thomaziv|camatcha){1}'
To ssh://gitlab.ensimag.fr/glapp2024/gl03.git
 ! [remote rejected] cette_branche_est_nulle -> cette_branche_est_nulle (pre-receive hook declined)
error: failed to push some refs to 'ssh://gitlab.ensimag.fr/glapp2024/gl03.git'
```

Figure 1: erreur nom de branche

Cela signifie que le nom de la branche est non valide. Vous pouvez fixer ça en renommant la branche courante avec cette commande:

```
git branch -m "Un_meilleur_nom_de_branche"
```

Une erreur fréquente est la longueur de la partie <titre> qui ne dois pas faire plus de 30 caractères
Privilégier une version abrégé du titre

`PGL-04/implement_operator_plus_and_minus/wendelsj`

`PGL-04/feat_op_plus_min/wendelsj`

Effectuer un commit

- N’oubliez pas de add vos changements

Verifier le status des fichiers add :

- Verifier le status de votre add avant de commit pour être sûr de n’avoir commit que la partie correspondant au commit que vous voulez faire

```
git status
git diff
```

Ou utiliser la GUI de vscode en checkant le diff de chaque fichier

Pour le commit, respecter le pattern suivant:

```
[PGL-##] type(scope): <message>
```

avec :

`type :`
`feat|fix|docs|refactor|test|build|perf|settings|remove|typo|other|decac|rename|move|copy|rebase|merge|settings`

`scope : context|codegen|syntax|test|lexer|git|maven|file|others|all|options`

- Pour les types settings, rebase et merge, le (scope) n’est pas obligatoire
- Il est possible de faire un revert d’un ancien commit. Dans ce cas le nom du commit devra être: Revert “<ancien nom de commit>”

- Par exemple:

```
git commit -m "[PGL-10] feat(context): blablabla"
```

Fournir des commits unitaires (pas de modif codegen et context dans le même commit par exemple)

En cas de push qui renvoie une erreur similaire:

```
remote: GitLab: Commit message does not follow the pattern '^(\[PGL-[0-9]+\]){1}\s(feat|fix|docs|refactor|test|b
uild|perf|settings|remove|typo){1}(\((context|codegen|syntax|test|lexer|git|maven|file)\)){0,1}(\:){1}\s(.{8,100
})'
To ssh://gitlab.ensimag.fr/glapp2024/gl03.git
 ! [remote rejected] PGL-0/DocGit/canaguyv -> PGL-0/DocGit/canaguyv (pre-receive hook declined)
error: failed to push some refs to 'ssh://gitlab.ensimag.fr/glapp2024/gl03.git'
```

Figure 2: erreur nom de commit

Il y a une petite astuce qui permet de remanier un peu ses commits. Tout d’abord, rentrez la commande:

```
git rebase -i HEAD~number
```

Avec number = le nombre de commit en partant du dernier que vous voulez remaniez.

Cela va vous ouvrir la fenêtre suivante dans le terminal:

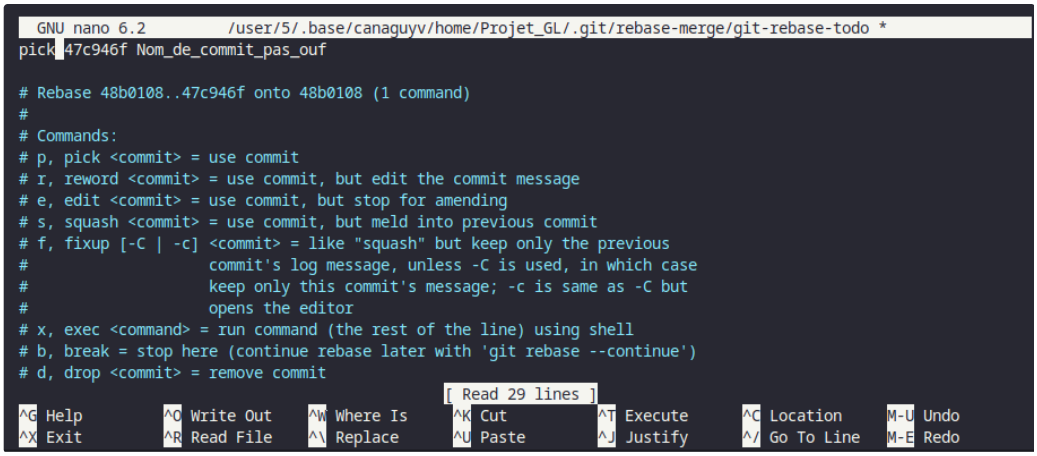


Figure 3: Remaniement des commits

Sur cette fenêtre vous devez remplacer les “pick” devant vos nom de commits de la façon suivante:

- Laisser **pick** pour ne pas toucher ce commit.
- Remplacer par **drop** pour supprimer un commit.
- Remplacer par **reword** pour éditer le nom du commit.

Les autres options vous ne devrez pas en avoir besoin.

Dans le cas du reword, vous devriez avoir (après ctrl+X pour quitter et Y puis entrée pour valider les modifs):

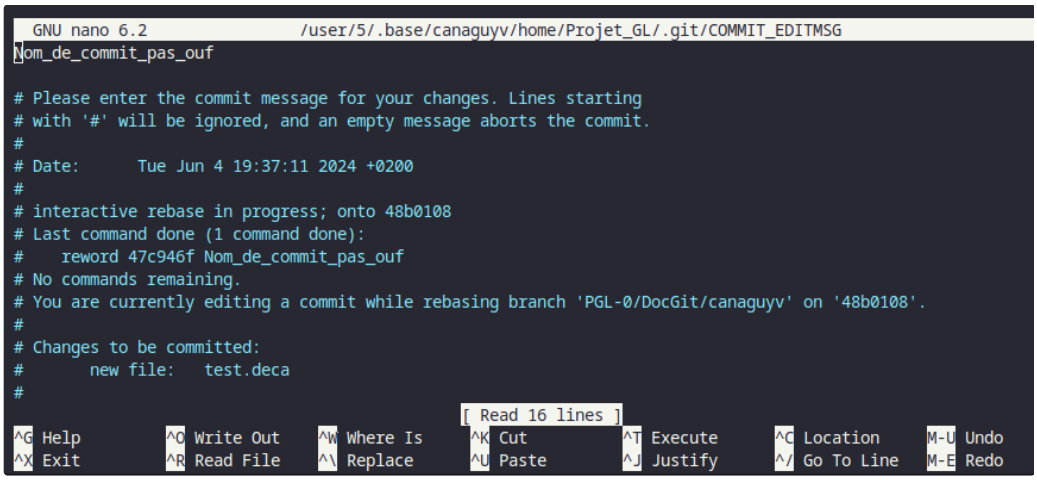


Figure 4: Rename Commit

Dans cette fenêtre, remplacez seulement l’ancien nom de commit par le nouveau et tout est bon ! (n’oublier pas : ctrl+X puis Y puis entrée pour valider les changements en fonction de votre éditeur (ici nano))

Gestion des MR:

Possible via vscode mais on n’a pas encore essayé donc à voir pour plus tard peut être.

Créer sa MR sur GitLab:

Aller sur le repo gitlab puis:

- Si vous avez push récemment, GitLab vous propose de créer une merge request dans un encadré vert en haut de la page principale.
- Sinon cliquer sur merge request à gauche de la page puis sur créer une merge request.

⚠

Normalement tout est automatique mais assurer vous que:

- La branche source est bien la vôtre.
- La branche destination est bien develop.
- L'option "delete source branch after merge" est bien coché

Ensuite remplissez la description en vous aidant du template (n'oubliez pas de remplacer les ## dans le lien pour avoir un lien vers le ticket Jira et de copier le lien de la mr dans le ticket jira).

Et voila MR créée.

Côté Reviewer:

⚠ Avant toute chose n'oubliez pas de respecter votre rôle (ex: les respo tests doivent être plus qu'attentif sur les tests).

Pour les reviewers vous pouvez cliquer sur les changes afin d'ouvrir tout les fichiers modifiés ainsi que leurs modifications.

Ensuite vous pouvez regarder le code et si vous détectez un erreur:

Cliquez sur cette icône sur la ligne qui vous semblent fausse:

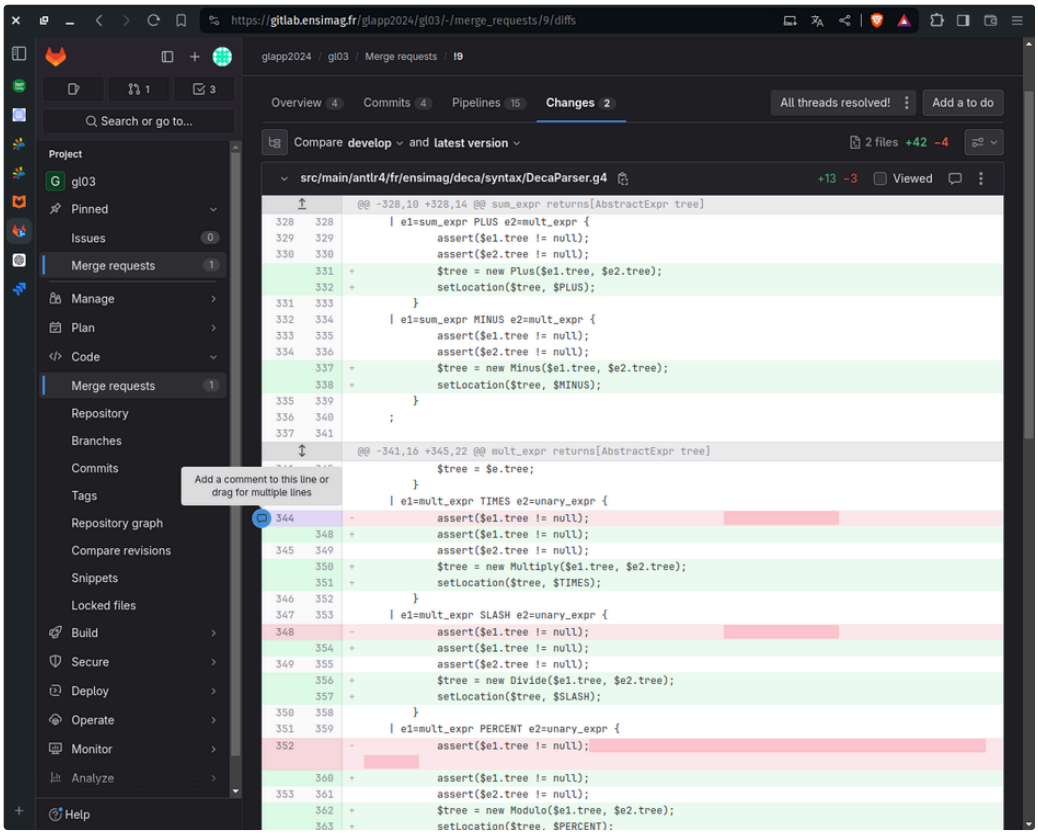


Figure 5: add comment

Ensuite rentrer votre commentaire puis cliquer sur start review pour bloquer la MR le temps que les commentaires soit résolu. Exemple:

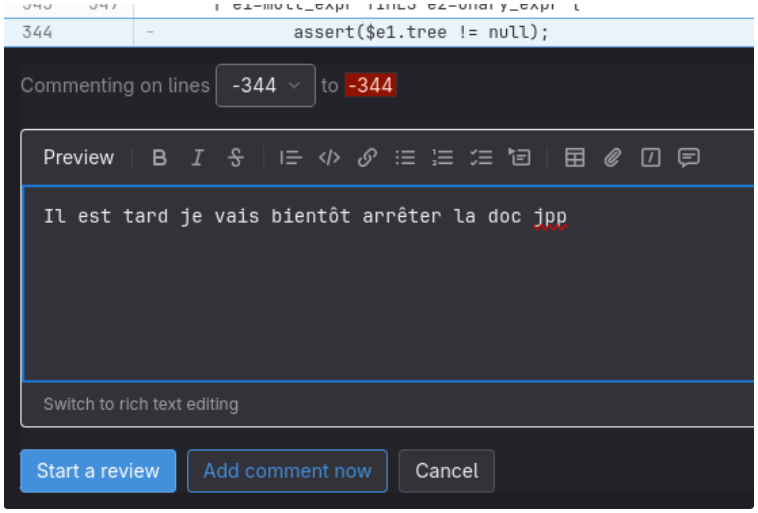


Figure 7: Start Review

Après que la correction soit faite et que le développeur vous la notifiez, vérifier que la correction vous satisfait puis fermer commentaire et review afin de débloquer la MR.

Enfin si tout est bon cliquer sur le gros bouton vert APPROVE sur l' overview de la MR.

Et voilà !

Côté dev qui ont malheureusement reçu un commentaire pour leur mauvais code

Rien de bien trop compliqué de votre côté il faut juste (Sur la bonne branche !!):

- Lire attentivement les commentaires
- Effectuer les modifications pour résoudre les commentaires
- Faire un beau commit
- Et push

Cela permettra de mettre à jour la MR automatiquement. Une fois que vous pensez avoir tous résolu, prévenez le reviewer à l'origine du commentaire qui le fermera si accepté.

Et Voilà !

Les règles d'approbation de MR (dans develop)

- 1 approval de testeur
- 1 approval de code owner
- 1 approval d'un développeur n'ayant pas les rôles précédents

Le pipeline CI/CD

Pipeline PASSED :

A chaque MR, le pipeline de test passent sur le code, cela execute tout les tests précédemment créer (pas encore les nouveaux)

Lorsque les tests réussissent, sur la MR vous verrez :

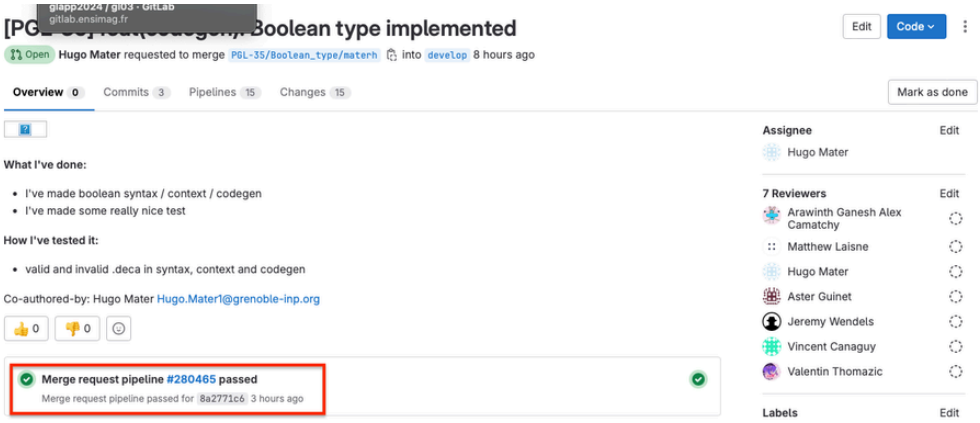


Figure 8: Pipeline CI passed

> ⚠ Cette condition est essentielle (obligatoire) pour pouvoir merge

Dans le cas où les tests passent, il n'y a pas de problèmes, si les autres conditions pour merge son remplie, le merge peut être fait.

Pipeline FAILED :

Si malheureusement vous avez fait tout planté et que les tests sont tout rouge, pas de panique, vous pouvez aller voir votre test en cliquant directement sur la MR sur le test, ou dans l'onglet `Jobs` dans le menu à gauche sur GitLab

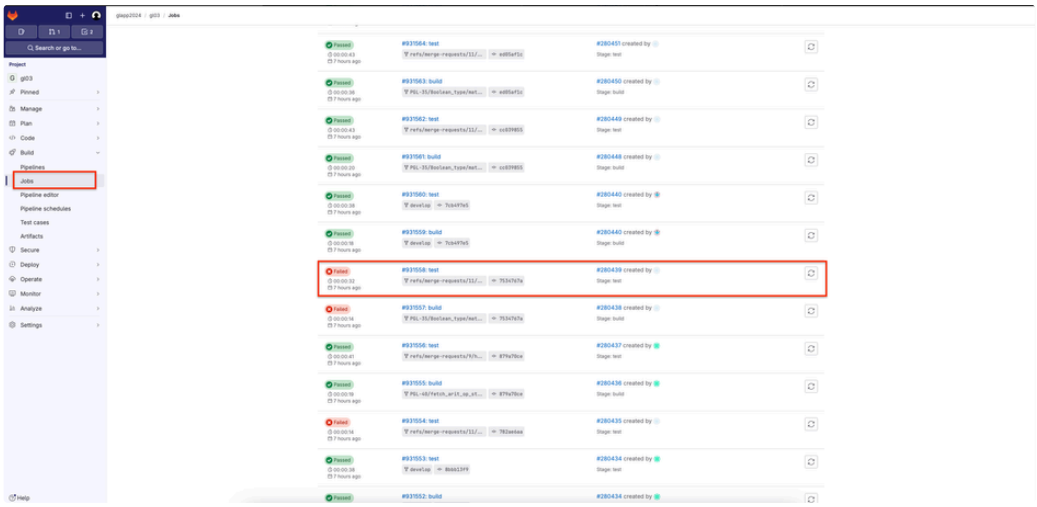


Figure 9: Pipeline CI FAILED

Et vous pouvez visualiser les logs du test qui a failed, et donc analyser la source du bug.

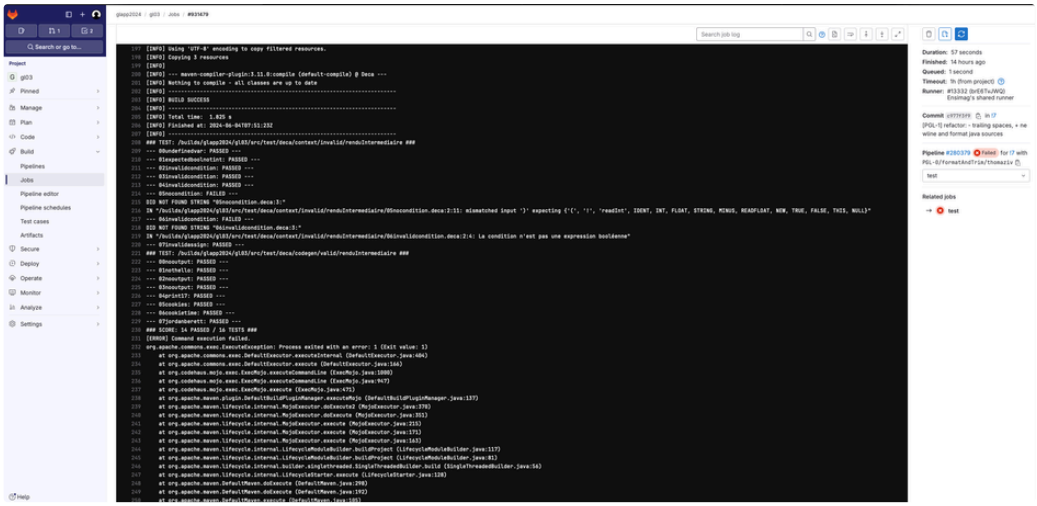


Figure 10: Pipeline CI Logs

Deux cas possibles :

- Il y a une regression
  Solution : debugger le code en analysant les logs pour trouver la source de la regression
- Un des nouveaux test implémentés dans votre MR plante
  Solution : corriger votre test

Warning important:

Si vous ne pouvez pas push ou pull à cause de conflits, c'est sûrement que votre branche local n'est pas à jour suite à un rebase ou autre dans la branche remote. Vous pouvez alors effectuer:

```
git pull --rebase
```

Si gitLab vous annonce des conflits lors d'un rebase vous devez gérer les conflits en local. Effectuer donc:

Attention, le push force effectué à la fin écrase l'historique de la branche sans retour en arrière possible. Ne suivez cette démarche que si:

- vous êtes seul sur votre branche et avez la dernière version de celle-ci
- vous êtes plusieurs, personnes n'a de modifs en attente, vous avez bien la dernière version de la branche ET TOUT LES CONTRIBUTEURS SONT AU COURANT ET D'ACCORD.

```
git checkout develop
git pull
git checkout votre_branche
git pull (Assurer vous d'avoir la dernière version de votreBranche)
git rebase develop --rebase-merge (Conserve l'historique des MR)
```

Ensuite gérer les conflits avant de faire un:

```
git push --force
```

Uniformisation de l'IDE:

- Rajouter à la racine du projet un .vscode/settings.json contenant:

```
{
    "editor.defaultFormatter": "redhat.java",
    "editor.formatOnSave": true,
    "editor.formatOnSaveMode": "file",
    "editor.codeActionsOnSave": {
        "source.organizeImports": "always"
    }
}
```

Cela permet de formatter le code à la sauvegarde d'un fichier (tabulation, import etc)

- Dans les settings du projet, déplacer la target dans le .vscode:

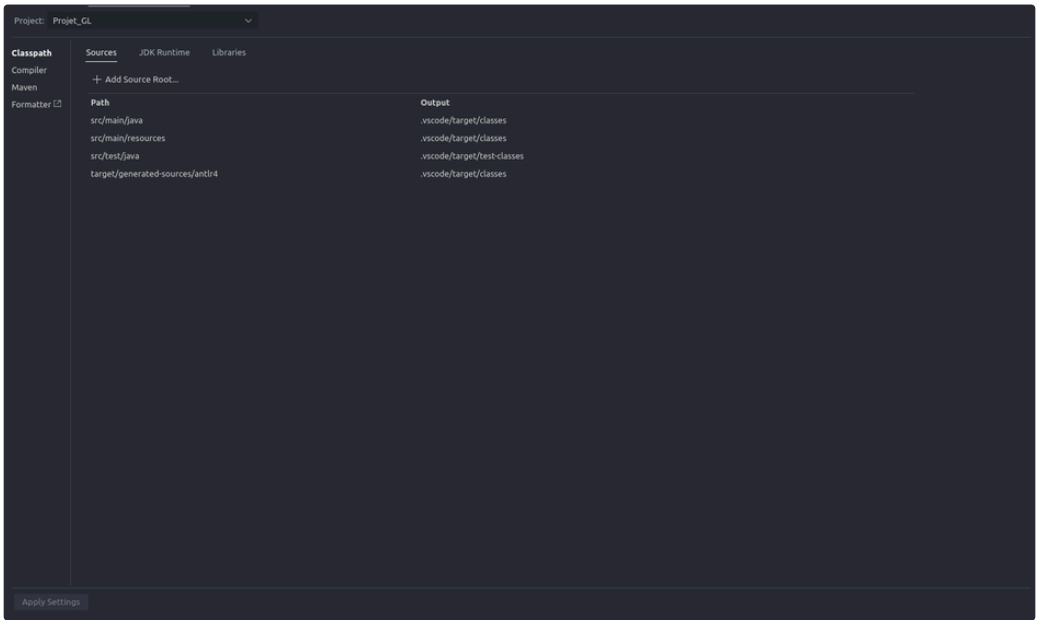


Figure 11: Settings project Java

Cela permet d'empêcher vscode de gérer lui même des erreurs de compilation ou autre afin d'exécuter maven comme si l'on était dans un terminal en dehors de vscode

Les manueltests ne sont donc plus fait automatiquement et après un mvn clean il faut bien faire un mvn test-compile et plus seulement mvn compile

- Dans le répertoire du projet, exécuter la commande suivante:

```
git config --local commit.template .gitmessage
```

Cela permet de configurer la gui vscode pour les commits afin d'avoir un template et donc de faciliter la création de commits.

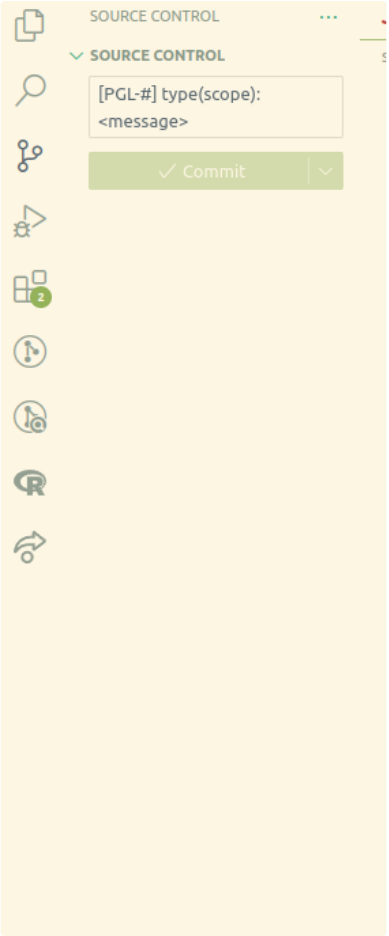


Figure 12: Template de commit intégré à vscode